How To: Maintain the Project

How To: Maintain the Project

Use this guide when you change the engine code, the app, or the docs, and need to keep everything else in sync. This project treats documentation as living: every code change gets a matching documentation pass in the same step, not a later one.

How to run the Python test suite

```
cd tests
python3 -m pytest -v
```

All 30 tests must pass before you ship a change. If a test fails, fix the code or the test — do not delete a failing test to make the suite pass.

How to check the JavaScript app matches the Python engine

The app (`app/index.html`) is a hand-written copy of the Python engine's logic. It is not generated from the Python code, so it needs a manual check after every engine change.

1. Extract the `<script>` block from `app/index.html` into a plain JavaScript file.
2. Run that file under Node.js.
3. Compare its output against the same hand-derived values used in `tests/test_fj_wager.py`, `tests/test_fj_bestresponse.py`, and `tests/test_dd_wager.py`.

This step is manual. There is no automated CI check yet — see risk #8 in `RISK_REGISTER.md`.

How to regenerate the PDF companion for a doc

Every Markdown doc in this project ships a PDF twin in `build/`. If you edit a `.md` file, regenerate its PDF in the same pass.

```
pandoc <source.md> -o build/<name>.pdf \
  -V geometry:margin=1in -V colorlinks=true -V linkcolor=blue \
  --highlight-style=tango \
  --pdf-engine=pdflatex \
  --pdf-engine-opt=-interaction=nonstopmode \
  --pdf-engine-opt=-halt-on-error \
  -V title="<Doc Title>"
```

The `--pdf-engine-opt` flags stop `pdflatex` from hanging on an error. Without them, a broken build waits forever for input that never comes.

If the build fails on a Unicode character error, search the source Markdown for the character named in the error and replace it with a plain ASCII equivalent (for example, a plain hyphen instead of a Unicode minus sign).

How to update the changelog and risk register

1. Add a new entry at the top of `CHANGELOG.md`, in Keep a Changelog format, with an ISO date (`YYYY-MM-DD`).
2. If the change resolves, changes, or adds a risk, update `RISK_REGISTER.md` in the same pass. Do not delete an old risk's row to mark it resolved — update its status column and explain why.
3. If the change reverses an earlier documented conclusion (in the white paper, an ADR, or a past changelog entry), add a note that states what changed and why. Do not silently rewrite the old conclusion.
4. Check that the version number at the top of `CHANGELOG.md` matches the version stamped in `docs/white-paper.md`.

How to check everything before you call a change "done"

Run this checklist, in order:

1. `python3 -m pytest -v` from `tests/` — all tests pass.
2. The JS/Python parity check above, if you touched the engine or the app.
3. `CHANGELOG.md`, `RISK_REGISTER.md`, and `docs/white-paper.md` updated, if the change affects behavior or a documented conclusion.
4. Every changed `.md` file has a regenerated PDF in `build/`.
5. The cross-project artifact audit script (`../portfolio/build/check_projects.py`) — it fails the check if a required artifact is missing or a version number does not match. As of this writing, that script's project list does not yet include this project; add an entry for it there before relying on the automated check.